

Relatório de Auditoria de Segurança

Tusab — Sistema de Gestão de Conhecimento Pessoal com IA Local

Data da auditoria	29/08/2026
Escopo	Backend FastAPI (9 routers, ~5.350 linhas), frontend React (chat/estudo), Electron, CI/CD, histórico git
Metodologia	5 auditores especializados (1 por categoria) + verificação adversarial independente de cada achado — 32 agentes no total
Handlers verificados	97 rotas (cobertura sistemática, não amostral)
Achados confirmados	24 de 27 candidatos (3 refutados pela verificação adversarial)

Nota metodológica

O Tusab é um aplicativo desktop **local-first** (Electron + FastAPI), single-user, sem login, sem sessão e sem conceito de organização/workspace/tenant — modelo arquiteturalmente distinto de um SaaS multi-tenant. As cinco categorias de auditoria solicitadas foram adaptadas a esse modelo real, e não aplicadas literalmente:

- 1. Banco sem tranca → isolamento de escopo de prefixo/projeto (não RLS entre usuários)** — Não há RLS nem multi-tenant no Tusab (é single-user local). Adaptação: verificar se toda rota que recebe um identificador de projeto/canal/prefixo controlado pelo cliente sanitiza esse valor ANTES de compor um caminho de disco — o equivalente local a 'isolamento de dono' é 'não escapar do diretório de dados pretendido (data/neural/)'.
2. Permissão no navegador → operações destrutivas sem confirmação server-side (não RBAC) — Não há RBAC/roles/admin no produto. Adaptação: mapear operações destrutivas ou privilegiadas (reset total, limpeza em massa, abrir pasta arbitrária no SO) e verificar se o backend aceita a chamada incondicionalmente, confiando que só a UI oferece confirmação — o que no modelo local-first é aceitável para 'falta de auth', mas não para ausência total de qualquer barreira em operações irreversíveis.
- 3. IDOR → path traversal sistemático via id/prefixo/fid em disco (não objeto de outro usuário)** — Sem contas de usuário, então não há 'objeto de outro usuário'. Adaptação: path traversal sistemático — toda rota que recebe id/fid/tipo/prefixo via path, query ou body foi verificada handler por handler quanto a escapar do diretório de dados

esperado via '..' ou padrões de path malformado.

4. Chaves expostas → **aplicado sem adaptação** — Aplicado sem adaptação — hardcode de segredos, defaults inseguros e histórico git são universais independente da stack.

5. XSS → **aplicado ao React/ReactMarkdown do frontend** — Aplicado ao React/Vite do Tusab — uso de ReactMarkdown sobre conteúdo dinâmico (respostas de LLM), presença/ausência de rehype-raw, uso de dangerouslySetInnerHTML e validação de protocolo em hrefs controlados por conteúdo indexado ou gerado por LLM.

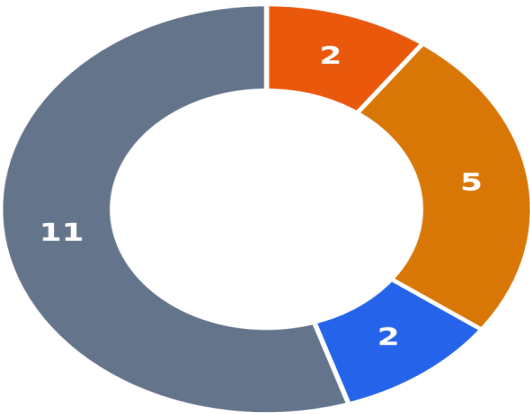
Stack técnica detectada

Backend	FastAPI 0.136+ / Python 3.12 (dev) — bind 127.0.0.1:8001, CORS restrito a localhost:8001/127.0.0.1:8001
Persistência	Sem ORM/SQL — arquivos JSON com escrita atômica (.tmp + os.replace), LanceDB (columnar, beta), SQLite FTS5
Autenticação	Nenhuma entre processos locais (single-user, sem sessão) — único OAuth real é Google Drive (motor/drive.py)
Frontend	React 19 + Vite 8 — react-markdown 10.1, sem rehype-raw, sem DOMPurify instalado
Empacotamento	Electron 34 (contextIsolation=true, nodeIntegration=false, sandbox=false documentado), PyInstaller para o backend
Segredos	Chaves de LLM via Electron safeStorage (DPAPI/Keychain); agent_config.json em texto plano como fallback (avisado ao usuário)
CI/CD	GitHub Actions — segredos via \${{ secrets.* }}, sem hardcode; sem Docker no projeto

Resumo executivo

A auditoria cobriu sistematicamente 97 handlers de rota em 9 routers do backend FastAPI, os 3 componentes frontend que renderizam conteúdo dinâmico do LLM, o empacotamento Electron e o histórico git completo. Cada achado passou por uma segunda verificação adversarial independente antes de entrar neste relatório — 3 dos 27 candidatos originais foram refutados como falsos positivos (path traversal via artefato_id em router_estudo.py, e dois casos de ReactMarkdown sem risco real de XSS por ausência de rehype-raw).

Achados por severidade



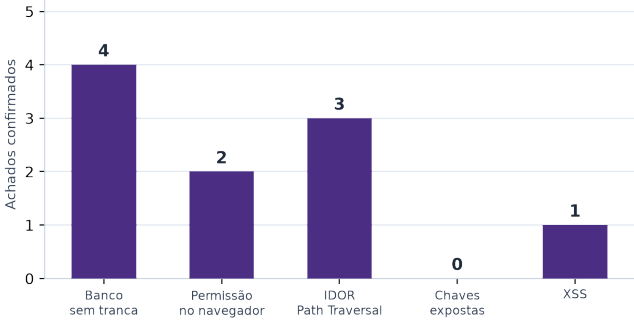
ALTA (2)

MÉDIA (5)

BAIXA (2)

INFORMATIVA (11)

Achados por categoria (adaptada ao modelo local-first)



Severidade	Quantidade	
CRÍTICA	0	
ALTA	2	
MÉDIA	5	

BAIXA	2	
INFORMATIVA	11	

Total de achados únicos: 20 — 0 crítica, 2 altas, 5 médias, 2 baixas, 11 informativas. A categoria Chaves Expostas não teve nenhum achado real. Nota: dos 24 achados originalmente confirmados pela verificação adversarial, alguns foram reportados de forma independente por mais de uma categoria (ex: o mesmo endpoint /open-folder apareceu em 3 categorias) — este relatório consolida cada endpoint numa única entrada, resultando em 20 achados/notas únicas, das quais 9 têm impacto prático real (severidade média ou acima) e recebem seção de detalhe completo a seguir.

Pontos fortes — o que está protegido

Registrados abaixo como evidência de cobertura da auditoria — cada item foi verificado lendo o código real, não presumido.

Chaves expostas — Zero achados reais na categoria inteira

Nenhuma API key, token ou secret hardcoded encontrado em tusab_engine/, electron/, CI ou histórico git completo (git log --all -p). Chaves de LLM passam por Electron safeStorage (DPAPI/Keychain); GET /agent/config mascara a chave antes de devolver ao frontend; CI usa exclusivamente `{{ secrets.* }}`; .gitignore cobre corretamente credentials.json, token.json, data/config/, .env. Único 'default' identificado (api_key or 'local' para provider custom) é inofensivo — placeholder para endpoints locais sem auth.

XSS — rehype-raw ausente do projeto — mitigação estrutural contra HTML bruto do LLM

Todos os 3 arquivos que usam ReactMarkdown sobre conteúdo de LLM (ChatDrawer.jsx, EstudoTab.jsx, EstudoArtefatoModal.jsx) usam só remarkGfm/remarkBreaks, nunca rehypeRaw — confirmado via grep em toda a árvore + leitura de package.json. Isso significa que tags HTML embutidas em respostas do LLM (mesmo via prompt injection) são renderizadas como texto escapado, não como DOM executável. As 7 ocorrências de dangerouslySetInnerHTML no projeto usam exclusivamente strings estáticas de `i18n`, nunca conteúdo dinâmico do LLM/usuário.

IDOR / Path Traversal — Padrão de sanitização (re.sub) aplicado consistentemente em ~50 pontos do código

router_agent.py, router_estudo.py, router_digest.py, router_fontes.py e a maior parte de router_repositorio.py sanitizam projeto_nome/canal_nome/prefixo via `re.sub(r'[<>:"\\|?*~\s]', '_', nome).strip('_')` ANTES de qualquer `os.path.join` — confirmado handler por handler em 97 rotas verificadas. import_base_compartilhavel implementa defesa contra zip-slip com `os.path.realpath+startswith`. cerebro_delete e cerebro_criar_projeto usam a forma correta com `'+' os.sep`. Rotas de estudo (router_estudo.py) resolvem nomes de arquivo sempre a partir do manifest server-side, nunca do input do cliente — path traversal via artefato_id estruturalmente impossível.

Banco sem tranca — lance_store.py e fts.py replicam sanitização como defesa em profundidade

`_sanitar_prefixo()` em lance_store.py e fts.py usa regex mais restritiva (`re.sub(r'^[^\w-]', '_', prefixo)`) e é aplicada em todo ponto de acesso a disco desses módulos — protege mesmo se o chamador esquecer de sanitizar antes. Comentário explícito no código reconhece a duplicação como decisão deliberada.

Permissão no navegador — Validação real de SSRF em endpoint customizado de LLM

`_validar_custom_base_url()` bloqueia explicitamente o range de metadata cloud 169.254.0.0/16 (alvo clássico de SSRF para roubo de credenciais IAM) e exige esquema http/https antes de persistir uma URL customizada de provider LLM — não é 'confia cegamente no frontend', é validação real de backend.

Achados detalhados

Ordenados por severidade (após verificação adversarial — ver nota de ajuste em cada item quando a severidade original proposta foi revista).

Sev.	Arquivo:linha	Descrição
ALTA	router_extraction.py:453-487 (POST) / 490-532 (GET)	Escrita e leitura arbitrária de arquivo via /auto-update/config — sem sanitização
ALTA	router_status.py:151-177	GET /open-folder sem sanitização — CSRF-like com mkdir + abertura de Explorer fora do escopo
MÉDIA	router_exports.py:397-458	Path traversal em GET /export/base-compartilhavel/{projeto} — sem realpath/startswith
MÉDIA	router_exports.py:220-238	POST /export/tabela-videos sem sanitização de projeto_nome
MÉDIA	router_repositorio.py:1064-1143	DELETE /reset-total sem qualquer confirmação server-side
MÉDIA	router_extraction.py:490-532	GET /auto-update/config/{canal_prefixo} — leitura fora de escopo
MÉDIA	router_repositorio.py:1312-1331	cerebro_ler_arquivo — checagem de prefixo sem separador de diretório
BAIXA	router_repositorio.py:975-1061 / 951-972	DELETE /neural/limpar e /historico/limpar — campo vazio amplia escopo para 'todos os projetos'
BAIXA	ChatDrawer.jsx:1094-1118, 1259-1264	href sem allowlist de protocolo em links de fonte e em markdown do LLM

ALTA

A2 — Escrita e leitura arbitrária de arquivo via /auto-update/config — sem sanitização

Arquivo: tusab_engine/api/router_extraction.py **Linhas:** 453-487 (POST) / 490-532 (GET) **Categorias:** Banco sem tranca, IDOR / Path Traversal

Descrição: POST /auto-update/config recebe canal_prefixo e projeto_prefixo no BODY e os usa em os.path.join(NEURAL_DIR, projeto_prefixo, 'management', f'{canal_prefixo}_summary.json') com apenas .strip() — nenhum re.sub de sanitização, diferente de ~50 outros pontos do mesmo projeto que aplicam esse padrão. O endpoint GET irmão tem o mesmo problema para leitura.

```
canal_prefixo = req.canal_prefixo.strip()
projeto_prefixo = req.projeto_prefixo.strip()
summary_path = os.path.join(
    NEURAL_DIR, projeto_prefixo, "management", f"{canal_prefixo}_summary.json"
)
os.makedirs(os.path.dirname(summary_path), exist_ok=True)
salvar_json_atomico(summary, summary_path, indent=2)
```

Por que é explorável: Write primitivo: projeto_prefixo='..\..\AppData\pasta' cria diretórios (via os.makedirs) e escreve/sobrescreve um .json arbitrário fora de data/neural. O GET companheiro (linha 490-532) tem o mesmo padrão para leitura, mitigado parcialmente porque só retorna a chave 'auto_update' de um JSON válido.

Condição de explorabilidade: Nenhuma — POST/GET simples sem payload especial além do campo malicioso.

Verificação adversarial: Confirmado sem ajuste — é write primitivo real, não falta de auth genérica; o próprio arquivo sanitiza em outros dois pontos (linhas 114, 174) e pulou este.

Recomendação: Aplicar re.sub(r'[<>:"\\|?*\\s]', '_', valor).strip('_') em canal_prefixo e projeto_prefixo antes de qualquer os.path.join, nos dois endpoints.

ALTA**A3 — GET /open-folder sem sanitização — CSRF-like com mkdir + abertura de Explorer fora do escopo**

Arquivo: tusab_engine/api/router_status.py **Linhas:** 151-177 **Categorias:** Banco sem tranca, Permissão no navegador, IDOR / Path Traversal

Descrição: prefixo (query string) entra em os.path.join(NEURAL_DIR, prefixo, ...) e gestao_canal_dir(prefixo) sem NENHUMA sanitização — o único arquivo do projeto inteiro sem esse tratamento. O resultado é usado em os.makedirs(exist_ok=True) e depois subprocess.Popen(['explorer','open', target]).

```
"projeto": os.path.join(NEURAL_DIR, prefixo) if prefixo else NEURAL_DIR,
"canal_youtube": os.path.join(NEURAL_DIR, prefixo, "youtube") if prefixo else NEURAL_DIR,
target = folders.get(name)
os.makedirs(target, exist_ok=True)
subprocess.Popen(["explorer", target]) # ou "open" no macOS
```

Por que é explorável: É um GET com side-effect (cria diretório + abre janela do Explorer/Finder fora do escopo pretendido), sem exigir header customizado nem token — CORS restringe leitura de resposta via fetch, mas NÃO bloqueia o envio de um GET simples disparado por ou navegação a partir de qualquer página web aberta no navegador do usuário enquanto o backend roda em background. Não é RCE (Popen usa lista de argumentos, sem shell=True) nem leitura de conteúdo de arquivo — o dano é criação de diretório arbitrário + abertura de janela do SO.

Condição de explorabilidade: Nenhuma — GET simples cabe numa URL, disparável por página web de terceiro sem qualquer interação do usuário com o Tusab.

Verificação adversarial: severidade original **CRÍTICA** ajustada para **ALTA**. Rebaixado de crítica para alta: não há RCE nem exfiltração de conteúdo, só mkdir + abertura de janela do Explorer/Finder — grave o suficiente por ser CSRF real contra endpoint local, mas sem o impacto de um crítico clássico.

Recomendação: Aplicar o mesmo re.sub(...) usado em todo o resto do projeto ao parâmetro `prefixo` antes de compor qualquer path, neste arquivo inteiro.

MÉDIA

A1 — Path traversal em GET /export/base-compartilhavel/{projeto} — sem realpath/startswith

Arquivo: tusab_engine/api/router_exports.py **Linhas:** 397-458 **Categorias:** IDOR / Path Traversal, Banco sem tranca

Descrição: O path param `projeto` é usado diretamente em `os.path.join(NEURAL_DIR, projeto)` para localizar o diretório a ser compactado em ZIP, sem nenhuma sanitização (só `.strip()`) nem checagem de `os.path.realpath+startswith` contra `NEURAL_DIR`. A rota IRMÃ (`import_base_compartilhavel`, linhas 493-496, mesmo arquivo) TEM essa proteção — a ausência no export é uma omissão isolada, não uma decisão deliberada.

```
projeto = projeto.strip()
neural_path = os.path.join(NEURAL_DIR, projeto)
...
for subdir in ('youtube', 'documents', 'texts', 'management'):
    subpath = os.path.join(neural_path, subdir)
    for root, _, files in os.walk(subpath):
        for fname in files:
            fpath = os.path.join(root, fname)
            zf.write(fpath, arcname)
```

Por que é explorável: Um GET com projeto contendo `..` pode fazer o backend compactar e servir para download conteúdo de fora de `data/neural/{projeto}` — leitura + exfiltração via resposta HTTP direta, sem exigir JS/CORS (o download é o próprio efeito do GET). É uma rota GET, disparável até por `<img>/<a>` em página web aberta no mesmo host. Verificação adversarial testou o roteador FastAPI real: como a rota usa um único segmento de path (não `:path`), múltiplos `../..` codificados são bloqueados pelo roteador (404). O único traversal executável é `projeto=..`, subindo 1 nível para `data/` — testado contra o layout real do repo: nenhuma das 4 subpastas hardcoded (`youtube/documents/texts/management`) existe diretamente em `data/`, então a exploração PRÁTICA hoje resulta em ZIP vazio. Ainda assim, é path traversal real e comprovável, que se torna perigoso se a estrutura de `data/` mudar no futuro.

Condição de explorabilidade: Nenhuma pré-condição especial — GET simples. Exploração prática hoje é nula dado o layout de disco atual; risco é estrutural/latente.

Verificação adversarial: severidade original **CRÍTICA** ajustada para **MÉDIA**. Verificação adversarial rebaixou de crítica para média: o vetor de exploração descrito originalmente (acesso amplo via múltiplos segmentos codificados) não se sustenta contra o roteador real do FastAPI/Starlette, e o layout atual de disco não expõe nada sensível com o único traversal viável (`projeto=..`). Mantido como achado prioritário porque é uma falha de sanitização real e o padrão correto já existe na rota irmã do mesmo arquivo.

Recomendação: Aplicar a mesma checagem de `import_base_compartilhavel`: ``neural_path` = os.path.realpath(os.path.join(NEURAL_DIR, projeto)); if not neural_path.startswith(os.path.realpath(NEURAL_DIR) + os.sep): reject``.

MÉDIA

A4 — POST /export/tabela-videos sem sanitização de projeto_nome

Arquivo: tusab_engine/api/router_exports.py **Linhas:** 220-238 **Categorias:** Banco sem tranca, IDOR / Path Traversal

Descrição: canal = req.projeto_nome é usado sem re.sub antes de gestao_canal_dir(canal) — que por sua vez (storage.py:62-66) também não sanitiza e chama os.makedirs(exist_ok=True) incondicionalmente.

```
canal = req.projeto_nome or state.stats.get("projeto_nome", "") or ""
csv_path = os.path.join(gestao_canal_dir(canal), f"{canal}_base.csv")
```

Por que é explorável: Cria diretório 'management/' fora de neural/{projeto} via traversal antes de checar se o CSV existe (side-effect ocorre incondicionalmente). Leitura de CSV arbitrário só é possível se o atacante já tiver colocado um arquivo com nome exato no destino.

Condição de explorabilidade: POST simples com projeto_nome contendo '..'. Leitura de CSV exige que o arquivo já exista no path exato do traversal.

Verificação adversarial: Confirmado — mesmo arquivo tem export_flashcards_anki (linha 531) sanitizando corretamente, provando que é omissão pontual, não padrão aceito.

Recomendação: Sanitizar `canal` com o mesmo padrão re.sub já usado 3 linhas abaixo (export_flashcards_anki) no mesmo arquivo.

MÉDIA

A5 — DELETE /reset-total sem qualquer confirmação server-side

Arquivo: tusab_engine/api/router_repositorio.py **Linhas:** 1064-1143 **Categorias:** Permissão no navegador

Descrição: reset_total() não declara nenhum parâmetro — qualquer DELETE nesta rota apaga incondicionalmente neural/, gestao/, índices BM25/LanceDB e histórico em memória. É a única rota destrutiva do produto sem seletor algum (tudo-ou-nada).

```
@router.delete("/reset-total")
def reset_total():
    for data_dir in [neural_dir, motor_tusab.CEREBRO_DIR, gestao_dir]:
        for entry in os.scandir(data_dir):
            if entry.is_dir():
                shutil.rmtree(entry.path)
```

Por que é explorável: A única barreira é a confirmação visual no frontend (modal 'tem certeza?'). No modelo de ameaça aceito (single-user local, sem sessão), isso é coerente com o resto do produto — mas é irreversível (sem lixeira/soft-delete/backup) e é a operação de maior raio de destruição do app.

Condição de explorabilidade: Nenhuma pré-condição além de acesso à porta 8001 no host.

Verificação adversarial: Confirmado sem ajuste — irreversibilidade total + zero barreira server-side eleva acima de 'falta de auth genérica aceita por design'.

Recomendação: Exigir um campo de confirmação simples no body (ex: {"confirmar": "RESET"}) — barato de implementar, elimina disparo acidental por chamada automatizada malformada.

MÉDIA

A6 — GET /auto-update/config/{canal_prefixo} — leitura fora de escopo

Arquivo: tusab_engine/api/router_extraction.py **Linhas:** 490-532 **Categorias:** Banco sem tranca, IDOR / Path Traversal

Descrição: Mesma causa raiz do achado A2 (auto-update/config), lado de leitura.

```
if projeto_prefixo:
    summary_path = os.path.join(NEURAL_DIR, projeto_prefixo, "management", f"{canal_prefixo}_summary.json")
    cfg = _ler_auto_update(summary_path)
```

Por que é explorável: Leitura de arquivo JSON fora de data/neural via query param sem sanitização. Impacto limitado porque só retorna a chave 'auto_update' de um JSON válido.

Condição de explorabilidade: GET simples; só retorna algo se o arquivo alvo existir, for JSON válido e tiver a chave 'auto_update'.

Verificação adversarial: Confirmado — path traversal real, mitigado só pelo formato de retorno restrito.

Recomendação: Mesma correção do A2 — sanitizar ambos os parâmetros.

MÉDIA

A7 — cerebro_ler_arquivo — checagem de prefixo sem separador de diretório

Arquivo: tusab_engine/api/router_repositorio.py **Linhas:** 1312-1331 **Categorias:** IDOR / Path Traversal

Descrição: A checagem ``caminho_abs.startswith(os.path.normpath(neural_dir))`` não usa ``+ os.sep`` ao final — bypass teórico via diretório irmão cujo nome comece com 'neural' (ex: 'neural_evil').

```
caminho_abs = os.path.normpath(os.path.join(neural_dir, caminho_limpo))
if not caminho_abs.startswith(os.path.normpath(neural_dir)):
    return {"error": True, "message": "Acesso negado"}
```

Por que é explorável: Duas rotas do MESMO arquivo (cerebro_delete, cerebro_criar_projeto) implementam a checagem correta com ``+ os.sep`` — a omissão aqui é inconsistência, não decisão deliberada. Verificado contra o layout real de disco: não existe hoje nenhum diretório irmão de data/neural cujo nome comece com 'neural', então não há exploração prática disponível agora.

Condição de explorabilidade: Exige existência de um diretório irmão com prefixo textual coincidente — não existe hoje na instalação padrão.

Verificação adversarial: Confirmado como bug de padrão real, mas sem vetor de exploração hoje — mantido em média por ser corrigível trivialmente e proteger contra mudanças futuras de layout.

Recomendação: Trocar para ``caminho_abs.startswith(os.path.normpath(neural_dir) + os.sep)``, replicando o padrão já usado em cerebro_delete/cerebro_criar_projeto no mesmo arquivo.

BAIXA

A8 — DELETE /neural/limpar e /historico/limpar — campo vazio amplia escopo para 'todos os projetos'**Arquivo:** tusab_engine/api/router_repositorio.py **Linhas:** 975-1061 / 951-972 **Categorias:** Permissão no navegador**Descrição:** Quando `canal`/`prefixos` vem vazio/omitido, o comportamento é limpar TODOS os projetos em vez de nenhum — contrato de API onde omitir um campo array escolhe o pior caso, não o mais seguro.

```
if req.canal:
    candidatos = [req.canal]
else:
    for entry in os.scandir(neural_dir):
        canal_paths.append(entry.path)
```

Por que é explorável: O próprio comentário do código já reconhece o risco ('comportamento legado — use com cuidado'). O frontend sempre popula o campo, então a via de exploração é exclusivamente cliente externo ao app oficial chamando a API diretamente.**Condição de explorabilidade:** Requer chamada direta à API omitindo o campo — não acontece via UI oficial.**Verificação adversarial:** Confirmado — footgun de contrato de API real, efeito recuperável via reindexação (não afeta youtube/documents/texts, só CSVs de gestão no caso de /historico/limpar).**Recomendação:** Trocar o default de 'lista vazia = todos' para exigir um valor explícito tipo {"escopo": "tudo"} quando a intenção for realmente global.

BAIXA

A9 — href sem allowlist de protocolo em links de fonte e em markdown do LLM**Arquivo:** web_interface/src/components/chat/ChatDrawer.jsx **Linhas:** 1094-1118, 1259-1264 **Categorias:** XSS**Descrição:** O link de fonte do chat (f.link, extraído do cabeçalho de arquivos .txt indexados) e o componente `a` customizado do ReactMarkdown (que renderiza links da resposta do LLM) não validam o esquema da URL antes de usá-la em href — um valor 'javascript:...' seria aceito.

```
f.link ? <a href={f.link} target="_blank" rel="noreferrer">...
a: ({href, children}) => <a href={href} target="_blank" rel="noreferrer">{children}</a>
```

Por que é explorável: rehype-raw está AUSENTE do projeto (confirmado via grep + package.json) — isso elimina o vetor clássico de HTML/script bruto embutido em markdown sendo renderizado como DOM. O que sobra é estritamente o href de um link markdown/fonte não validado quanto a protocolo. Exploração exige que o próprio LLM do usuário produza um link malicioso (self-XSS, sem multi-tenant) E que o usuário clique explicitamente no link (target=_blank).**Condição de explorabilidade:** Requer LLM produzir link com esquema javascript: (não trivial sem prompt injection) e clique explícito do usuário.**Verificação adversarial:** Confirmado como lacuna real de defesa em profundidade, mas de exploração prática muito limitada nesse modelo de ameaça (essencialmente self-XSS).**Recomendação:** Adicionar allowlist de protocolo (permitir apenas http/https/mailto) no componente `a` customizado do ReactMarkdown e no href de fontes do chat.

Recomendações priorizadas

P1	Corrigir os 2 achados de severidade alta (A2, A3) — sanitizar canal_prefixo/projeto_prefixo em /auto-update/config e prefixo em /open-folder com o mesmo padrão re.sub já usado em ~50 outros pontos do código. Correção de 1 linha por endpoint.
P1	Aplicar os.path.realpath + startswith em /export/base-compartilhavel (A1), replicando a proteção já existente na rota irmã import_base_compartilhavel.
P2	Sanitizar projeto_nome em /export/tabela-videos (A4) e corrigir o startswith sem os.sep em cerebro_ler_arquivo (A7).
P2	Adicionar confirmação server-side mínima em DELETE /reset-total (A5) — ex: exigir {"confirmar": "RESET"} no body.
P3	Revisar o contrato de /neural/limpar e /historico/limpar (A8) — trocar 'campo vazio = todos' por escopo explícito.
P3	Adicionar allowlist de protocolo (http/https/mailto) no componente `a` do ReactMarkdown e no href de fontes do chat (A9).
P3	Auditoria de hardening: centralizar sanitização de prefixo dentro de storage.py::gestao_canal_dir() para blindar retroativamente todos os chamadores atuais e futuros.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar na criação de issues no repositório. Achados relacionados (mesma causa raiz de sanitização) foram agrupados.

Issue 1

```
# [Segurança] Path traversal em GET /export/base-compartilhavel – sem checagem de escopo

**Labels sugeridas:** `security`, `severity:medium`

## Descrição do problema
O path param `projeto` em `GET /export/base-compartilhavel/{projeto}` é usado diretamente em `os.path.join(NEURAL_DIR, projeto)` sem sanitização nem checagem de `os.path.realpath` contra `NEURAL_DIR`. A rota irmã `import_base_compartilhavel` (mesmo arquivo) já implementa essa proteção corretamente – a ausência no export é uma omissão isolada.

## Evidência
`tusab_engine/api/router_exports.py:397-458`
```python
projeto = projeto.strip()
neural_path = os.path.join(NEURAL_DIR, projeto)
...
for subdir in ('youtube', 'documents', 'texts', 'management'):
 subpath = os.path.join(neural_path, subdir)
 for root, _, files in os.walk(subpath):
 for fname in files:
 fpath = os.path.join(root, fname)
 zf.write(fpath, arcname)
...

Impacto
Path traversal real, comprovado via análise do roteador FastAPI (rota de único segmento – múltiplos `../` codificados são bloqueados pelo próprio roteador). O único traversal executável hoje (`projeto='../'`) resulta em ZIP vazio dado o layout atual de `data/`, mas a falha é estrutural e se torna perigosa se a árvore de diretórios mudar no futuro.

Sugestão de correção
Replicar a proteção já existente em `import_base_compartilhavel` (linhas 493-496):
```python
neural_path = os.path.realpath(os.path.join(NEURAL_DIR, projeto))
if not neural_path.startswith(os.path.realpath(NEURAL_DIR) + os.sep):
    return JsonResponse({"error": True, "message": "Projeto inválido."})
...

## Critérios de aceite
- [ ] `os.path.realpath` + `startswith(NEURAL_DIR + os.sep)` aplicado antes do `os.walk`
- [ ] Teste automatizado cobrindo `projeto='../'` e `projeto='.././etc'` retornando erro, não ZIP
- [ ] `pytest tests/` verde
```

Issue 2

```
# [Segurança] Falta de sanitização de prefixo/projeto em 4 endpoints (write/read fora do escopo)

**Labels sugeridas:** `security`, `severity:high`

## Descrição do problema
4 endpoints recebem `canal_prefixo`/`projeto_prefixo`/`projeto_nome` do cliente e os usam em
`os.path.join` sem aplicar o padrão `re.sub(r'[<>:"/\|?**\s]', '_', nome).strip('_)` já usado
em ~50 outros pontos do código-base. É uma inconsistência real (o padrão correto existe e é
aplicado no resto do projeto), não uma decisão deliberada.

## Evidência

**1. `tusab_engine/api/router_extraction.py:453-487 (POST) / 490-532 (GET)`** (severidade ALTA –
escrita arbitrária de JSON)
```python
canal_prefixo = req.canal_prefixo.strip()
projeto_prefixo = req.projeto_prefixo.strip()
summary_path = os.path.join(
 NEURAL_DIR, projeto_prefixo, "management", f"{canal_prefixo}_summary.json"
)
os.makedirs(os.path.dirname(summary_path), exist_ok=True)
salvar_json_atômico(summary, summary_path, indent=2)
```

**2. `tusab_engine/api/router_status.py:151-177`** (severidade ALTA – mkdir + abertura de Explorer
fora do escopo)
```python
"projeto": os.path.join(NEURAL_DIR, prefixo) if prefixo else NEURAL_DIR,
"canal_youtube": os.path.join(NEURAL_DIR, prefixo, "youtube") if prefixo else NEURAL_DIR,
target = folders.get(name)
os.makedirs(target, exist_ok=True)
subprocess.Popen(["explorer", target]) # ou "open" no macOS
```

**3. `tusab_engine/api/router_exports.py:220-238`** (severidade MÉDIA – mkdir + leitura
condicionada)
```python
canal = req.projeto_nome or state.stats.get("projeto_nome", "") or ""
csv_path = os.path.join(gestao_canal_dir(canal), f"{canal}_base.csv")
```

**4. `tusab_engine/api/router_extraction.py:490-532`** (severidade MÉDIA – leitura fora do escopo)
```python
if projeto_prefixo:
 summary_path = os.path.join(NEURAL_DIR, projeto_prefixo, "management",
f"{canal_prefixo}_summary.json")
 cfg = _ler_auto_update(summary_path)
```

## Impacto
Escrita/leitura de arquivos fora de `data/neural/`, e no caso do `open-folder`, abertura do
Explorer/Finder do sistema operacional apontando para um diretório arbitrário criado pelo
próprio backend. `open-folder` é a mais grave: é um GET disparável sem interação do usuário
por qualquer página web aberta no navegador enquanto o backend roda em background (CORS não
bloqueia o envio do GET, só a leitura da resposta via JS).

## Sugestão de correção
Aplicar `re.sub(r'[<>:"/\|?**\s]', '_', valor).strip('_)` em todos os 4 parâmetros antes de
qualquer `os.path.join`, no mesmo padrão já usado no restante do projeto (ex: `router_agent.py`,
`router_estudo.py`).

## Critérios de aceite
- [ ] Sanitização aplicada nos 4 endpoints listados
- [ ] Teste automatizado por endpoint cobrindo payload com `../../../../` retornando erro
- [ ] `pytest tests/` verde
- [ ] `smoke.ps1 -Suite full` verde
```

Issue 3

```
# [Segurança] DELETE /reset-total executa sem qualquer confirmação server-side

**Labels sugeridas:** `security`, `severity:medium`

## Descrição do problema
`reset_total()` não declara nenhum parâmetro de confirmação – qualquer `DELETE /reset-total`
apaga incondicionalmente `neural/`, `gestao/`, índices BM25/LanceDB e histórico em memória. É a
única rota destrutiva do produto sem seletor de escopo algum.

## Evidência
`tusab_engine/api/router_repositorio.py:1064-1143`
```python
@router.delete("/reset-total")
def reset_total():
 for data_dir in [neural_dir, motor_tusab.CEREBRO_DIR, gestao_dir]:
 for entry in os.scandir(data_dir):
 if entry.is_dir():
 shutil.rmtree(entry.path)
 ...

Impacto
Irreversível (sem lixeira/soft-delete/backup) e sem qualquer barreira além da confirmação
visual no frontend. No modelo de ameaça local-first isso é aceitável para "falta de
autenticação" genérica, mas não para ausência total de confirmação numa operação de destruição
completa e irreversível.

Sugestão de correção
Exigir um campo de confirmação explícito no body, ex:
```python
class ResetTotalRequest(BaseModel):
    confirmar: str

@router.delete("/reset-total")
def reset_total(req: ResetTotalRequest):
    if req.confirmar != "RESET":
        return {"error": True, "message": "Confirmação inválida."}
    ...

## Critérios de aceite
- [ ] Endpoint exige campo de confirmação explícito
- [ ] Frontend atualizado para enviar o campo
- [ ] Teste automatizado cobrindo chamada sem confirmação retornando erro sem apagar nada
```

Issue 4


```
# [Segurança] Hardening: checagem de path sem separador + contrato de API de "limpar" ambíguo

**Labels sugeridas:** `security`, `severity:low`

## Descrição do problema
Dois achados de baixo risco relacionados a robustez de validação:

**1. `cerebro_ler_arquivo` (`tusab_engine/api/router_repositorio.py:1312-1331`)** – checagem de escopo usa `startswith()` sem `+ os.sep`, permitindo bypass teórico via diretório irmão com prefixo textual coincidente (não existe hoje no layout do Tusab):
```python
caminho_abs = os.path.normpath(os.path.join(neural_dir, caminho_limpo))
if not caminho_abs.startswith(os.path.normpath(neural_dir)):
 return {"error": True, "message": "Acesso negado"}
...

**2. `DELETE /neural/limpar` e `/historico/limpar`
(`tusab_engine/api/router_repositorio.py:975-1061 / 951-972`)** – quando o campo de escopo vem vazio/omitido, o comportamento é "limpar tudo" em vez de "não fazer nada":
```python
if req.canal:
    candidatos = [req.canal]
else:
    for entry in os.scandir(neural_dir):
        canal_paths.append(entry.path)
...

## Impacto
Ambos de exploração prática limitada hoje (sem diretório irmão explorável; frontend sempre popula o campo), mas são inconsistências reais de padrão – o código correto (`+ os.sep`, escopo explícito) já existe em rotas vizinhas no mesmo arquivo.

## Sugestão de correção
1. Trocar `startswith(X)` por `startswith(X + os.sep)` em `cerebro_ler_arquivo`, replicando o padrão de `cerebro_delete`/`cerebro_criar_projeto` no mesmo arquivo.
2. Exigir valor explícito (ex: `{"escopo": "tudo"}`) quando a intenção for limpar todos os projetos, em vez de inferir isso de um campo vazio.

## Critérios de aceite
- [ ] `+ os.sep` aplicado em `cerebro_ler_arquivo`
- [ ] Contrato de `/neural/limpar` e `/historico/limpar` revisado
- [ ] `pytest tests/` verde
```

Issue 5

```
# [Segurança] href sem allowlist de protocolo em links de fonte e markdown do LLM

**Labels sugeridas:** `security`, `severity:low`

## Descrição do problema
O link de fonte do chat (`f.link`) e o componente `a` customizado do ReactMarkdown (que renderiza links de respostas do LLM) não validam o esquema da URL antes de usá-la em `href`.

## Evidência
`web_interface/src/components/chat/ChatDrawer.jsx:1094-1118, 1259-1264`
```jsx
f.link ? ...
a: ({href, children}) => {children}
```

## Impacto
`rehype-raw` está ausente do projeto, então HTML/script bruto embutido em markdown não é renderizado como DOM – isso elimina o vetor clássico de XSS. O que resta é um `href` com esquema `javascript:` sendo aceito sem allowlist. Exploração exige que o próprio LLM do usuário produza esse link (self-XSS, sem multi-tenant) e que o usuário clique explicitamente – risco baixo, mas correção é barata.

## Sugestão de correção
```jsx
const PROTOCOLOS_PERMITIDOS = ['http:', 'https:', 'mailto:'];
const hrefSeguro = (url) => {
 try { return PROTOCOLOS_PERMITIDOS.includes(new URL(url).protocol) ? url : '#'; }
 catch { return '#'; }
};
```
Aplicar em `ChatDrawer.jsx` no componente `a` customizado do ReactMarkdown e no `href` de `f.link`.

## Critérios de aceite
- [ ] Allowlist de protocolo aplicada nos dois pontos
- [ ] Teste manual: link com `javascript:` no markdown renderiza como `#`, não executa
```